

TALLER 2

Algoritmos de Búsqueda & Optimización de Colonias de Hormigas (ACO)

Integrante	Actividad asignada
Lucy Barreno	P1 – Algoritmos de búsqueda en laberintos (BFS / DFS)
Darwin Simba	P2 – ACO: Actividades A, B y C
Ana Haro	P2 – ACO: Actividades D y E
Jhon Del Castillo	Ensayo – Modelos de lenguaje y algoritmos de búsqueda

Repositorio GitHub (Pull Request para merge al main): <https://github.com/Borreguin/USFQ-Wshop/pull/52>

1. Algoritmos de Búsqueda en Laberintos — Lucy Barreno

Descripción general

Lucy implementó la Parte 1 del taller, cuyo objetivo es resolver cuatro laberintos usando algoritmos clásicos de búsqueda. El laberinto se representa como una matriz de caracteres donde '#' indica pared, el espacio vacío indica camino libre, 'S' es el punto de inicio y 'E' la meta.

Representación como grafo

Cada celda libre del laberinto se convierte en un nodo de un grafo implementado mediante lista de adyacencia (diccionario de Python). Solo se consideran movimientos en cuatro direcciones: arriba, abajo, izquierda y derecha. Las paredes no forman parte del grafo.

Algoritmos implementados

BFS (Breadth-First Search): Explora el grafo por niveles garantizando el camino más corto. Utiliza una cola (deque) y registra nodos visitados y sus padres para reconstruir la ruta.

DFS (Depth-First Search): Explora en profundidad usando una pila. Es más rápido en algunos casos pero no garantiza la optimalidad del camino encontrado.

Resultados comparativos

Laberinto	Algoritmo	Long. camino	Nodos explorados	Tiempo (s)
1	BFS	15	43	0.0001
1	DFS	31	31	0.00007
2	BFS	45	158	0.0003
2	DFS	57	166	0.00027
3	BFS	345	1268	0.0029

3	DFS	453	1048	0.0023
4	BFS	473	2649	0.0056
4	DFS	723	1570	0.0042

BFS garantiza la ruta más corta; DFS suele ser más rápido pero produce caminos más largos. A mayor tamaño del laberinto, el costo computacional de BFS crece significativamente.

2. Optimización de Colonias de Hormigas (ACO) A–C — Darwin Simba

A. Ejecución y análisis de la implementación base

Darwin ejecutó el caso de estudio 1 del algoritmo ACO planteado en el repositorio. Se identificó que el algoritmo original presentaba un problema crítico: en ocasiones registraba como 'mejor camino' rutas que no alcanzaban el destino, ya que solo ordenaba todos los caminos por longitud sin verificar si el último nodo era el objetivo.

B. Corrección del caso de estudio 2

El bug fue corregido filtrando únicamente los caminos válidos (aquellos cuyo último elemento es el nodo destino) antes de seleccionar el más corto. Si ninguna hormiga llega al destino en una iteración, se omite la actualización de feromonas para esa iteración, evitando que el sistema 'aprenda errores'. Esto mejora la convergencia del algoritmo notablemente.

C. Descripción de parámetros del modelo

Parámetro	Descripción	Efecto de incrementarlo
num_ants	Número de hormigas (soluciones) por iteración	Más exploración, mayor costo computacional
evaporation_rate	Velocidad a la que desaparecen las feromonas	Olvida caminos rápido, mayor diversificación
alpha (α)	Importancia de las feromonas acumuladas	Refuerza caminos ya explorados (explotación)
beta (β)	Importancia de la heurística (distancia al destino)	Comportamiento más greedy hacia el objetivo
pheromones	Memoria colectiva del sistema	Guía a las hormigas hacia mejores soluciones

3. Optimización de Colonias de Hormigas (ACO) D–E — Ana Haro

D. Random Search y Grid Search aplicados al ACO

Grid Search evalúa exhaustivamente todas las combinaciones posibles dentro de una grilla predefinida de hiperparámetros. Garantiza encontrar la mejor combinación dentro del espacio definido, pero su costo computacional crece exponencialmente (maldición de la dimensionalidad).

Random Search selecciona aleatoriamente un número fijo de configuraciones dentro del espacio de búsqueda. Bergstra & Bengio (2012) demostraron que supera a Grid Search en la mayoría de casos, ya que explora más valores distintos del parámetro más influyente.

Para este ACO, **Grid Search es más adecuado** porque el espacio de hiperparámetros es pequeño (4 parámetros con rangos reducidos), el costo de evaluación es bajo, y permite explorar exhaustivamente interacciones críticas como alpha vs. beta en laberintos de cuadrícula 10×10.

E. ACO para el Travelling Salesman Problem (TSP)

El TSP es el problema canónico para el que ACO fue diseñado originalmente (Dorigo, 1991). En esta formulación, cada ciudad es un vértice de un grafo completo y cada arista tiene como heurística $\eta = 1/d$ (inverso de la distancia). Las hormigas construyen tours completos usando una lista tabú para no repetir ciudades (ciclo hamiltoniano válido). Los pasos de implementación son: (1) inicialización de feromonas uniformes; (2) construcción de rutas con regla probabilística $P(i \rightarrow j) = (\tau^\alpha \cdot \eta^\beta) / \Sigma$; (3) evaluación de distancia total; (4) evaporación $\tau \leftarrow (1-\rho) \cdot \tau$ y depósito $\Delta\tau = Q/L$; (5) repetición hasta criterio de parada, conservando siempre la mejor solución global.

4. Ensayo: LLMs y Algoritmos de Búsqueda — Jhon Del Castillo

Pregunta central

¿Funcionan los modelos de lenguaje (LLMs) como algoritmos de búsqueda? La pregunta surge de la intuición de que, al generar texto, un LLM parece 'elegir' entre alternativas de forma similar a como BFS o DFS exploran un grafo. El ensayo analiza críticamente esta analogía.

Arquitectura Transformer y generación de texto

Un LLM basado en Transformer (Vaswani et al., 2017) es un aproximador de distribuciones de probabilidad condicional: dado un contexto, estima la distribución del siguiente token mediante mecanismos de self-attention (Query, Key, Value), capas feed-forward apiladas y una proyección final sobre el vocabulario con softmax. La generación es autorregresiva: token a token, de izquierda a derecha, sin estado explícito ni capacidad de retroceder.

Similitudes superficiales y diferencias fundamentales

Aspecto	Algoritmo de búsqueda	LLM
Estado explícito	Sí — sabe dónde está y qué exploró	No — solo el contexto de entrada
Backtracking	Sí — puede retroceder y comparar	No — generación unidireccional
Garantías	Optimalidad o completitud según el algoritmo	Ninguna — predice texto probable
Determinismo	Determinista dado un estado inicial	Estocástico (temperatura de muestreo)
Objetivo	Resolver un problema definido	Predecir el siguiente token coherente

La única conexión real está en el entrenamiento: el ajuste de parámetros por gradient descent puede verse como búsqueda en el espacio de configuraciones para minimizar la función de pérdida. Técnicas como chain-of-thought prompting acercan la generación a un razonamiento más deliberado, pero el mecanismo subyacente sigue siendo predicción de tokens, no exploración de un grafo de estados.

